

Laboratorio di Ingegneria Informatica

Alessandro Chitarrini¹, Matteo Crugliano² e Davide Gaglione³

Sapienza Università di Roma

chitarrini.2129549@studenti.uniroma1.it,

crugliano.2127105@studenti.uniroma1.it, gaglione.2150498@studenti.uniroma1.it

1 Obiettivo del progetto

Il progetto mira a realizzare un sistema multi-agente che risponde a domande in linguaggio naturale attraverso l'uso di varie tecnologie come: Python, FastAPI, LangGraph, Ollama, Neo4j e Docker. Un orchestratore centrale interpreta la domanda e la delega a tre agenti specializzati: uno interroga knowledge graph, uno interroga api pubbliche per i dati in tempo reale e uno scompone le richieste di pianificazione in piani strutturati. Tutti i modelli sono eseguiti in locale tramite Ollama, senza dipendere da servizi a pagamento.

2 Organizzazione del codice

2.1 Setup

Dopo aver stabilito gli obiettivi generali e aver installato le librerie necessarie abbiamo iniziato una repository Git per garantire una fluida collaborazione. Ogni agente è un micro-servizio autonomo, con il proprio `Dockerfile`, le proprie dipendenze e la propria suite di test, così da poter essere sviluppato in parallelo. La struttura finale è la seguente:

```
sapienza-agents/  
├── shared/  
├── ui/  
├── orchestrator/  
├── agents/  
│   ├── kg/  
│   ├── planner/  
│   └── multiapi/  
└── docker-compose.yml
```

2.2 Orchestratore

L'orchestratore è realizzato con LangGraph ed è il punto di ingresso del sistema. Un nodo

supervisor classifica la domanda e sceglie quali agenti attivare; quelli selezionati vengono interrogati in parallelo e i loro risultati confluiscono in un nodo `synthesizer`, che li integra in un'unica risposta in linguaggio naturale. Le domande fuori scope vengono deviate su un nodo dedicato, senza sprecare chiamate agli agenti.

I nodi sono generati a partire da un registro in configurazione, quindi aggiungere un agente non richiede modifiche al grafo. Il fallimento di un agente non interrompe l'esecuzione: l'errore viene registrato e passato comunque al sintetizzatore. L'orchestratore si occupa infine della lingua, perché l'agente `kg` lavora in inglese: la domanda viene tradotta prima di essergli inoltrata e la risposta finale viene riportata nella lingua originale.

2.3 Agente `kg`

L'agente `kg` traduce una domanda in linguaggio naturale in una query da eseguire su un knowledge graph. La pipeline si articola in cinque passi: *entity linking*, recupero dello schema, traduzione, esecuzione e *grounding* dei risultati.

Le menzioni vengono estratte con GLiNER e associate alle entità del grafo. Serve poi lo schema, cioè l'elenco delle proprietà utilizzabili: quelle di Wikidata e DBpedia sono migliaia e non entrano in un prompt, quindi le più rilevanti vengono selezionate con una ricerca semantica su indice FAISS. Lo schema di un grafo Neo4j è invece piccolo e chiuso, e viene letto per intero senza costruire alcun indice. Se la query solleva un errore il modello la rigenera leggendo il messaggio dell'endpoint; se invece è valida ma non restituisce risultati, un secondo tentativo rilassa i filtri superflui e riformula la domanda.

L'architettura è organizzata per *provider*: ogni knowledge graph compone i propri componenti dietro interfacce comuni e si seleziona

¹Matricola: 2129549

²Matricola: 2127105

³Matricola: 2150498

per singola richiesta. Sono supportati Wikidata e DBpedia in SPARQL e un'istanza locale Neo4j in Cypher sul dominio cinema. Le generazioni sono affidate a due modelli distinti: `qwen2.5-coder:7b` per la scrittura della query, che è un compito di programmazione, e `qwen2.5:7b-instruct` per il linking, dove il modello generico disambigua meglio.

2.4 Agente planner

L'agente planner è un microservizio specializzato nello scomporre richieste in linguaggio naturale in piani temporali strutturati (formato JSON). Classifica la richiesta in uno di tre domini operativi (`study`, `travel`, `routine`) e delega a un dominio `unknown` il compito di respingere le richieste fuori scope; oltre a generare un piano da zero, supporta l'intento di *replanning* per eseguire aggiornamenti mirati su agende esistenti.

Prima della generazione, un modulo di *context gathering* decide se e come arricchire il piano con dati reali interrogando gli altri due agenti. Questo processo può avvenire tramite configurazione deterministica oppure in modalità *ReAct*, dove il modello decide autonomamente, passo per passo, quali tool chiamare. Il sistema è progettato per garantire resilienza: i fallimenti dei servizi esterni (es. timeout o tool non raggiungibili) non interrompono l'esecuzione, ma vengono registrati esplicitamente all'interno di apposite `contingency_notes`, mitigando eventuali allucinazioni senza far scendere a zero la *confidence* del piano.

L'output generato viene sottoposto a un rigoroso validatore logico e strutturale che verifica il rispetto dello schema e l'assenza di violazioni temporali (es. durate negative o sovrapposizioni di orari), innescando cicli iterativi di autocorrezione in caso di errore. Infine, il planner dispone di una funzionalità di *auto-discovery* per elencare dinamicamente i modelli disponibili. È l'unico agente della catena a supportare, oltre ai modelli locali (Ollama), anche provider cloud (Gemini, OpenRouter), garantendo un ripiego automatico (*fallback*) in locale qualora le chiavi API manchino o le chiamate esterne falliscano.

2.5 Agente multiapi

L'agente multiapi risponde interrogando api pubbliche. Un primo prompt classifica l'in-

tento della domanda, un secondo ne estrae i parametri e il provider corrispondente effettua la chiamata; il modello non scrive mai la risposta, che è ricavata dai soli dati restituiti dall'api. Gli intenti supportati sono quattro: `weather` (Open-Meteo, con distinzione fra condizioni correnti e previsioni fino a sei giorni), `exchange_rate` (Frankfurter, con dati storici fino al 1999), `country_info` (countries.dev) e `time_info` (world-time-api3); una domanda che non vi ricade è classificata `unknown` e produce un errore esplicito invece di una risposta inventata. Il classificatore riconosce anche i temi secondari, quindi una domanda che ne cita due riceve un risultato per ciascuno, fino a un massimo di due, mostrati in schede separate. Ogni risultato affianca ai campi grezzi dell'api una sintesi in linguaggio naturale, che orchestratore e planner usano come evidenza al posto del dizionario di campi tecnici. Le risposte sono messe in cache con una scadenza proporzionata a quanto invecchia il dato: l'ora locale non viene mai riusata, perché un orario in cache è per definizione sbagliato, mentre il meteo dura dieci minuti e i dati di un paese un giorno.

2.6 Web UI e containerizzazione

L'interfaccia web è una pagina statica servita da nginx, dalla quale si può interrogare l'orchestratore oppure un singolo agente. Oltre alla risposta finale mostra le evidenze intermedie: la query generata, i risultati grezzi e la confidenza. Il sistema è diviso in sei container orchestrati da `docker-compose.yml`: Neo4j, i tre agenti sulle porte `8000`, `8001` e `8002`, l'orchestratore sulla `8080` e l'interfaccia sulla `3000`. L'avvio è ordinato tramite dipendenze e controlli di salute, i container comunicano su una rete interna dedicata e raggiungono Ollama sull'host, mentre indici e cache dei modelli sono montati come volumi per non essere ricostruiti ad ogni riavvio.

3 Valutazione

Ogni agente è valutato con una metodologia adatta al proprio compito: l'agente kg su benchmark pubblici, il planner su un *golden dataset* personalizzato.

3.1 Agente kg

La valutazione usa due benchmark pubblici della famiglia QALD: QALD-10 su Wikidata

(394 domande) e lo split di test di QALD-9-plus su DBpedia (150 domande). La metrica è la **macro-F1** con le convenzioni QALD, che confronta l'insieme delle risposte prodotte con quello delle risposte attese. Il punteggio è riportato anche per tipo di domanda: booleane (*ask*), conteggi (*count*), domande risolvibili con un solo predicato (*single*) e domande che ne richiedono più di uno (*multi*).

Le risposte di riferimento non sono quelle registrate nel dataset ma vengono rigenerate eseguendo le query *gold* al momento della valutazione, perché i dataset sono invecchiati: su QALD-9-plus 35 domande su 150 non hanno più alcuna risposta registrata, e su un riferimento vuoto la convenzione QALD assegnerebbe il punteggio pieno anche a una pipeline fallita. Le domande prive di riferimento vengono escluse, ed è per questo che i totali valutati sono inferiori a quelli nominali.

Per capire se il knowledge graph aggiunga accuratezza o soltanto verificabilità, abbiamo posto le stesse domande allo stesso modello senza alcun accesso al grafo (*closed-book*).

	QALD-10		QALD-9-plus	
	pipe	c-b	pipe	c-b
domande	378/394		104/150	
macro-F1	0.377	0.284	0.205	0.224
ask	0.426	0.623	0.750	0.250
count	0.218	0.126	0.250	0.250
single	0.590	0.217	0.253	0.261
multi	0.225	0.297	0.117	0.187

pipe = pipeline completa, c-b = baseline closed-book

Su Wikidata il grafo guadagna un terzo di macro-F1 rispetto al modello nudo, ma quasi tutto il vantaggio viene dalle domande a un solo predicato. Su DBpedia il confronto si ribalta e il grafo aggiunge soltanto verificabilità: il valore del *grounding* dipende quindi dal grafo interrogato, non dall'architettura.

Il punteggio complessivo è basso ma il sistema è accurato quando risponde: su QALD-10 solo 197 query su 378 restituiscono righe, e su quelle il F1 medio è 0.621. Il problema non è quindi la qualità della traduzione ma il fatto che in metà dei casi la query, pur essendo valida, non trova nulla. Analizzando questi fallimenti la proprietà scelta è sbagliata nel 79% dei casi

e l'entità solo nel 67%: il collo di bottiglia è la scelta del predicato, non l'entity linking.

Abbiamo infine condotto alcuni esperimenti di ablazione per giustificare singole scelte di progetto. Il più significativo riguarda la disambiguazione delle entità: interpellare l'lm su ogni menzione non compra accuratezza rispetto a un semplice ranking di segnali (0.883 contro 0.890), e costa una chiamata in più per menzione.

3.2 Agente planner

L'agente planner è valutato su un *golden dataset* multi-dominio, inizialmente composto da 10 casi base e successivamente espanso a 30 casi per testare a fondo i modelli più promettenti. I 20 test aggiuntivi introducono vincoli stringenti come calcoli temporali complessi (*time_math*) e agende irrealizzabili (*impossible_schedule*), per misurare la robustezza dell'agente. In totale, la nuova suite conta 230 esecuzioni su 13 modelli (5 valutati sulla suite completa, 8 su quella ridotta).

Alle metriche strutturali (percentuale di successo nei domini supportati, correttezza di classificazione e tasso *zero-shot*) si affianca la valutazione qualitativa *llm-as-a-judge* su scala da 1 a 5, che giudica l'aderenza alla richiesta, l'ancoraggio al contesto, la fattibilità e la granularità. Come giudice è stato utilizzato *gemma-4-31b* in cloud, poichè è stato considerato come il modello più "potente" e con meno possibilità di crash dovuti dalla mole di controlli da eseguire.

Modello	Test	Succ.	Dom.	Sem.	0-shot
gemini-3.6-flash	30	1.00	1.00	4.80	0.88
gemini-3.5-f.-lite	30	1.00	1.00	4.63	0.80
nemotron-3-ultra	30	0.92	0.90	4.65	0.76
laguna-s-2.1	30	0.80	0.73	4.38	0.64
llama3.2	30	0.80	0.87	3.56	0.64
dots-3-note-prev.*	10	1.00	1.00	4.72	0.75
qwen3:1.7b*	10	1.00	1.00	3.17	1.00

Succ. = successo nei domini supportati, Dom. = accuratezza di dominio, Sem. = score semantico (1-5), 0-shot = tasso zero-shot. *Valutati solo sui 10 test base.

L'introduzione dei casi complessi ha evidenziato in modo chiaro i limiti di alcune architetture. Mentre sui test base il sistema non andava mai in crash (0%), la maggiore difficoltà ha innalzato il tasso di crash generale al 3.48%, trainato dai cedimenti di *laguna-s-2.1* (23.33% di crash rate sui nuovi casi, dovuti

unicamente a fallimenti delle chiamate all'API del provider e non all'incapacità del modello di risolvere l'intento) e dai timeout di esecuzione che hanno colpito llama3.2 nei casi più complessi. Inoltre, l'errore di validazione più comune è cambiato: non più la sovrapposizione degli orari, ma il calcolo errato delle durate (`duration_minutes_non_valido`), a dimostrazione delle crescenti difficoltà nel ragionamento aritmetico. Di conseguenza, il tasso di fallimento dei cicli di autocorrezione è salito dal 20% al quasi 31%.

Il confronto fra modelli evidenzia come i provider *cloud* (`gemini-3.6-flash` e `gemini-3.5-flash-lite`) mantengano un tasso di successo del 100% assorbendo bene la maggiore complessità, registrando solo una lieve, fisiologica flessione nello score semantico complessivo (da 5.00 a 4.80 per il modello top di gamma) per via dei vincoli più severi. Al contrario, llama3.2, che sui test base risultava il miglior compromesso in locale, ha mostrato forti incertezze sui casi difficili: il successo è sceso all'80% e la qualità semantica è crollata a 3.56, con un tasso di validazione salito al 32%. Modelli locali estremamente compatti come `qwen3:1.7b`, pur riuscendo a superare infallibilmente i test più semplici in modalità *zero-shot*, lo fanno a scapito di una semantica molto povera e schematica (3.17).

3.3 Agente multiapi

L'agente multiapi non ha un benchmark quantitativo: una volta classificato l'intento ed estratti i parametri, il suo compito si riduce a una chiamata http il cui esito dipende dall'api esterna. La verifica è quindi affidata alla suite di test, 139 in tutto, divisa in test di unità che girano senza rete su risposte finte, sia delle api sia del modello, e test di integrazione che interrogano le api reali e vengono saltati automaticamente quando il servizio non è raggiungibile o la chiave non è configurata.

4 Conclusioni

Il sistema integra tre agenti eterogenei dietro un unico orchestratore, e la valutazione mostra che i limiti si concentrano nell'agente kg, dove tradurre una domanda in una query su un'ontologia di migliaia di proprietà resta il compito più difficile. Le misure indicano però con precisione dove intervenire: la traduzione

è accurata quando produce una risposta, e il punteggio è limitato dai casi in cui la query è valida ma sceglie il predicato sbagliato.